

Mission 1

Inscription, connexion et profil

Guide de compréhension, cartographie des fichiers et vérification de conformité

Conclusion de l'audit : le code actuel couvre toutes les exigences fonctionnelles de la Mission 1. Les tests automatisés et le build réussissent. La capture Network réelle reste à réaliser localement avec MongoDB Atlas accessible.

Élément	Résultat vérifié
Tests frontend	4 tests réussis sur 4
Build Angular	Réussi - bundle initial 301,46 kB brut
Tests backend	2 tests réussis sur 2
Contrat HTTP	Routes et formats conservés
Rapport IA	Non modifié pour cette demande, conformément à la consigne

Document conçu pour être lu avant la restitution orale du TP1.

1. Ce que fait la partie utilisateur

La partie utilisateur transforme un visiteur non identifié en utilisateur authentifié. Une fois connecté, le navigateur conserve un JWT, Angular maintient un état réactif de l'utilisateur et les requêtes privées transportent une preuve d'authentification.

Répartition des responsabilités

Couche	Responsabilité	Fichiers principaux
Interface Angular	Formulaires, messages, boutons et redirections.	components/login-page, register-page, profile-page, app
Service Angular	Centralise les appels HTTP et l'état d'authentification.	shared/services/auth.service.ts
Infrastructure Angular	Ajoute le JWT, traite les 401 et protège les routes.	auth.interceptor.ts, auth.guard.ts, routes.ts, main.ts
API Express	Valide les données, vérifie le JWT et accède à MongoDB.	backend/src/app.js
Modèle MongoDB	Valide l'utilisateur, hache et vérifie le mot de passe.	backend/src/models/User.js

Règle d'architecture

Les composants n'injectent jamais HttpClient. Ils injectent AuthService, qui est la seule couche frontend chargée des appels d'authentification et de profil.

Chemin général

Composant Angular → AuthService → HttpClient → authInterceptor → proxy /api → API Express → modèle User → MongoDB

2. Tableau de conformité de la Mission 1

Exigence	Statut	Preuve dans le projet
Formulaires réactifs	OK	FormGroup/FormControl dans login-page.ts:17 et register-page.ts:18
Validations et erreurs	OK	Validators, markAllAsTouched, signaux error et messages dans les templates
POST register/login	OK	AuthService.register lignes 21-24; login lignes 15-18
Sauvegarde JWT	OK	localStorage gpc_token dans auth.service.ts:45-48; token jamais logué
Signal currentUser	OK	Déclaré ligne 12; mis à jour après auth, lecture et modification
Redirections	OK	Login vers /tracks; inscription vers /profile
Déconnexion	OK	AppComponent.logout puis AuthService.logout nettoie token et utilisateur
GET /users/me	OK	ProfilePage.load → AuthService.profile
PUT /users/me	OK	ProfilePage.save → AuthService.update
Gestion du 401	OK	authInterceptor: logout et navigation /login lignes 18-25
Séparation interface/service/API	OK	Aucun composant utilisateur n'appelle HttpClient directement
inject()	OK	Utilisé dans composants, service, guard et intercepteur

Point d'attention

Le guard vérifie seulement qu'un token existe localement. Il ne prouve pas que ce token est valide. La vraie validation se fait dans le middleware backend avec `jwt.verify`. Si le serveur répond 401, l'intercepteur nettoie la session et renvoie vers la connexion.

3. Flux complet d'une connexion

Lorsque l'utilisateur clique sur « Se connecter », le trajet est le suivant :

Étape	Action	Fichier
1	Le formulaire déclenche submit().	login-page.html → login-page.ts:28
2	Le composant vérifie le formulaire et bloque une double soumission.	login-page.ts:29-34
3	Le composant appelle auth.login(email, password).	login-page.ts:35-36
4	AuthService exécute POST /api/auth/login.	auth.service.ts:15-18
5	L'intercepteur reconnaît une route publique et n'ajoute pas de JWT.	auth.interceptor.ts:12-16
6	Express cherche l'utilisateur et vérifie le mot de passe haché.	backend/src/app.js:195-215; models/User.js
7	Le serveur renvoie { token, user } si les identifiants sont corrects.	backend/src/app.js:214-215
8	AuthService stocke le JWT et actualise token/currentUser.	auth.service.ts:45-48
9	Le composant redirige vers /tracks.	login-page.ts:37-40

Schéma annoté

Utilisateur	LoginPage	AuthService	API Express	MongoDB
clic	valide le formulaire	POST /auth/login	vérifie les identifiants	lit User
← redirection	← réponse succès	← stocke JWT + User	← {token,user}	← résultat

Le mot de passe transite uniquement dans le corps HTTPS en production. Il n'est ni stocké dans Angular ni placé dans le JWT. Le backend conserve un hash bcrypt, pas le mot de passe en clair.

4. Inscription, déconnexion et 401

Inscription

- Frontend : RegisterPage valide nom (2 caractères), email et mot de passe (8 caractères).
- Service : POST /api/auth/register avec {name, email, password}.
- Backend : valide les champs, refuse un email déjà utilisé, crée User, signe un JWT.
- Succès : AuthService met à jour localStorage, token et currentUser, puis navigation vers /profile.

Déconnexion

Le bouton est affiché dans app.html lorsque auth.token() existe. Il appelle AppComponent.logout(), qui appelle le service, supprime gpc_token, remet les deux Signals à null, puis navigue vers /login.

Jeton invalide ou expiré

1. Une requête privée reçoit le header Authorization. 2. Le backend exécute jwt.verify. 3. Si la signature ou l'expiration est invalide, il renvoie 401. 4. authInterceptor intercepte le 401. 5. Il appelle logout() et redirige vers /login.

Guard et intercepteur ne jouent pas le même rôle

Mécanisme	Avant ou après la requête ?	Rôle
authGuard	Avant l'ouverture d'une page	Évite d'ouvrir /profile ou /tracks sans token local.
authInterceptor	Pendant les requêtes HTTP	Ajoute le JWT aux routes privées et réagit aux 401.
middleware auth backend	À la réception côté serveur	Vérifie réellement la signature et l'expiration.

5. Profil utilisateur et mise à jour du nom

Lecture du profil

L'accès à /profile est protégé par authGuard. Le constructeur de ProfilePageComponent appelle load(). Cette méthode appelle AuthService.profile(), qui fait GET /api/users/me. La réponse met à jour currentUser et remplit le champ name.

Où s'effectue exactement la mise à jour ?

Côté	Fichier et méthode	Responsabilité
Frontend - interface	profile-page.html	Formulaire du nouveau nom et bouton Enregistrer.
Frontend - composant	profile-page.ts, save() lignes 41-60	Valide, appelle le service, affiche succès ou erreur.
Frontend - service	auth.service.ts, update() lignes 33-36	PUT /api/users/me avec {name}; actualise currentUser.
Frontend - intercepteur	auth.interceptor.ts	Ajoute Authorization: Bearer [masqué].
Backend - route	backend/src/app.js lignes 249-267	Cherche par req.auth.sub, modifie name et renvoie User.
Backend - modèle	backend/src/models/User.js	Impose notamment le nom requis et la longueur minimale.

Flux de modification

formulaire → ProfilePage.save() → AuthService.update(name) → PUT /api/users/me → authInterceptor → middleware auth → User.findByIdAndUpdate() → réponse User → currentUser.set(user) → interface réactive

Pourquoi le Signal est important ici ?

Après la réponse PUT, currentUser.set(user) rend immédiatement le nouveau nom disponible aux composants qui lisent ce Signal. Il n'est pas nécessaire de recharger toute la page.

6. Routes backend utilisées pendant le TP1

Méthode et URL	Publique ?	Corps / header	Réponse attendue
POST /api/auth/register	Oui	{name,email,password}	201 {token,user}; 400; 409
POST /api/auth/login	Oui	{email,password}	200 {token,user}; 401
GET /api/users/me	Non	Authorization: Bearer [masqué]	200 User; 401; 404
PUT /api/users/me	Non	{name} + Authorization	200 User; 400; 401; 404

Routes Angular associées

URL navigateur	Composant	Protection
/login	LoginPageComponent	Publique
/register	RegisterPageComponent	Publique
/profile	ProfilePageComponent	authGuard
/tracks	TracksPageComponent	authGuard

Important : une route Angular et une route API ne sont pas la même chose. /profile choisit une page dans le navigateur; /api/users/me demande des données au serveur.

7. Signal et localStorage : différence

Aspect	Signal Angular	localStorage
Nature	État réactif en mémoire.	Stockage persistant du navigateur.
Durée	Disparaît au rechargement.	Survit au rechargement et à la fermeture.
Réactivité	L'interface réagit automatiquement.	Aucune réactivité Angular automatique.
Usage du projet	token et currentUser pilotent l'interface.	gpc_token restaure la session.
Contenu	User public et token en mémoire.	Seulement le JWT, jamais le mot de passe.

Au démarrage, le Signal token est initialisé depuis `localStorage.getItem("gpc_token")`. Cela relie persistance et réactivité. En revanche, `currentUser` démarre à null : la page Profil appelle `GET /api/users/me` pour récupérer les données fiables du serveur.

Sécurité à connaître

- Le contenu d'un JWT est encodé et signé, pas nécessairement chiffré.
- Ne jamais y mettre un mot de passe ou un secret.
- Un token dans localStorage peut être lu par du JavaScript exécuté dans la page : il faut donc prévenir les failles XSS.
- La présence locale du token ne remplace jamais la vérification backend.

8. Checkpoint Network à réaliser localement

Cette preuve doit venir d'un vrai navigateur connecté à votre backend. Dans DevTools → Network → Fetch/XHR, réalisez les trois scénarios ci-dessous. Avant toute capture, masquez la valeur du JWT et le mot de passe.

Scénario	Méthode / URL	Statut	Authorization	À vérifier
Connexion réussie	POST /api/auth/login	200	Absent	Réponse {token,user}; masquer token et corps sensible.
Connexion refusée	POST /api/auth/login	401	Absent	Réponse Identifiants incorrects; erreur visible.
Lecture profil	GET /api/users/me	200	Bearer présent, valeur masquée	Réponse User public, sans passwordHash.
Ou modification	PUT /api/users/me	200	Bearer présent, valeur masquée	Corps {name}; réponse User actualisée.

Procédure de capture sûre

- Ouvrir Network, activer Preserve log si nécessaire, puis filtrer avec auth ou users/me.
- Sélectionner la requête et relever méthode, URL, statut, corps JSON, réponse et présence du header.
- Ne jamais montrer la valeur du header Authorization.
- Ne jamais capturer le mot de passe présent dans Request Payload.
- Recadrer la capture sur les informations non sensibles, puis l'ajouter au projet.

Limite de la vérification automatique

Les tests et le build réussissent, mais une capture Network réelle n'a pas été produite par l'agent : MongoDB Atlas n'était pas joignable depuis son environnement et aucun navigateur pilotable n'était disponible. Cette preuve doit donc être réalisée par le binôme.

9. Assistant IA, modèle et tokens

Quel modèle est utilisé ici ?

Cette session est un agent Codex fondé sur GPT-5. L'interface ne me fournit pas nécessairement l'identifiant exact du déploiement ou de la variante dans une forme que je puisse certifier. Pour un rapport académique, la formulation prudente est : « assistant Codex d'OpenAI, fondé sur GPT-5; variante exacte non exposée dans la session ».

Comment connaître la consommation de tokens ?

Dans une intégration API, la réponse fournit les compteurs d'usage, notamment les tokens d'entrée et de sortie. OpenAI fournit aussi un endpoint de comptage des tokens d'entrée : POST /v1/responses/input_tokens. Dans cette interface Codex, je n'ai pas accès à un compteur exact et vérifiable de tous les tokens consommés par la conversation; il faut consulter les informations d'usage exposées par le produit ou l'administrateur du workspace.

Qui conseille le meilleur modèle ?

La documentation officielle OpenAI propose un guide interactif de sélection. Son principe : choisir selon la difficulté, la qualité attendue, la latence et le coût, puis comparer sur les mêmes tâches. Elle présente Luna pour les tâches cadrées et fréquentes, Sol pour le travail quotidien demandant du jugement, et Astra pour les problèmes ambigus et les livrables exigeants.

Besoin	Choix de départ
Petite correction bien délimitée, extraction simple	Luna, effort faible ou moyen
Développement courant, recherche et vérification sérieuse	Sol, effort moyen
Analyse complexe, large contexte, livrable très exigeant	Astra, effort moyen ou élevé

Sources officielles

OpenAI - Model selection : <https://developers.openai.com/api/docs/guides/model-selection>

OpenAI - Counting tokens : <https://developers.openai.com/api/docs/guides/token-counting>

10. Questions de restitution orale

Question	Réponse courte attendue
Pourquoi passer par AuthService ?	Pour séparer l'interface, l'état utilisateur et le transport HTTP, et éviter de dupliquer les appels.
Où est stocké le JWT ?	Persisté sous gpc_token dans localStorage et reflété par un Signal token.
Qui ajoute Authorization ?	authInterceptor, seulement pour les requêtes non publiques.
Qui valide réellement le JWT ?	Le middleware auth du backend avec jwt.verify.
Pourquoi le guard ne suffit-il pas ?	Il teste seulement la présence locale du token; le navigateur est contrôlé par l'utilisateur.
Comment currentUser est-il actualisé ?	Avec tap() après login/register/profile/update, puis currentUser.set(...).
Que se passe-t-il sur 401 ?	L'intercepteur appelle logout(), nettoie la session et navigue vers /login.
Où le nom est-il mis à jour ?	ProfilePage.save → AuthService.update → PUT /api/users/me → User.findByIdAndUpdate.
Signal ou localStorage ?	Signal = mémoire réactive; localStorage = persistance non réactive.

Verdict final

Conforme côté code : oui. Tests : frontend 4/4, backend 2/2, build Angular réussi. Livrable restant au binôme : effectuer et ajouter une capture Network réelle, soigneusement anonymisée, lorsque MongoDB Atlas est accessible.

Fichiers de référence principaux

frontend-starter/src/app/components/{login-page,register-page,profile-page,app} •
 frontend-starter/src/app/shared/services/auth.service.ts • frontend-starter/src/app/shared/interceptors/auth.interceptor.ts •
 frontend-starter/src/app/shared/guards/auth.guard.ts • frontend-starter/src/app/routes.ts • frontend-starter/src/main.ts •
 backend/src/app.js • backend/src/models/User.js